Q29: Is AI Controllable? Agentic Workflows and Safety Protocols

DLS 2026 — Draft module

October 2026

Learning objectives

Students will distinguish model behavior from system control, map an agent's authority and trust boundaries, apply least privilege and approval gates, explain why prompt instructions alone are not access control, and create a bounded evaluation for a tool-using workflow.

1 Control is a systems property

No single prompt, benchmark, or safety classifier makes an agent controllable in every context. An agentic system combines a model, prompts, tools, credentials, memory, retrieval sources, workflow code, and human operators. Control therefore means that this assembled system behaves within specified boundaries under declared conditions, and that failures are detected, limited, and recoverable.

Start with a task contract. What outcomes are permitted? Which actions are irreversible, expensive, private, regulated, or safety-critical? Which tools can be called, with which arguments, under whose credentials? Who can stop the workflow? A useful control claim is bounded: "this agent may draft a response but cannot send it," or "this agent may propose a configuration change but a human approves the final write." It is stronger and more useful than "the model is safe."

2 Least privilege and authority boundaries

Least privilege gives each component only the permissions needed for its role. Separate read from write; separate staging from production; scope credentials to a service, resource, and expiration; and make high-impact tools require an authorization token that the model cannot forge. A tool protocol can standardize interfaces, but its existence does not make a server trustworthy; the host application must decide which servers, capabilities, and credentials to expose [1].

Approval gates should be semantic, not decorative. A system must verify an approver identity and bind approval to the proposed action, target, parameters, and expiry. A generic instruction such as "approved" is weak because it can be replayed or copied into another context. The included code demonstrates a small allowlist checker: it accepts only a request that matches an explicit approved tuple and rejects near matches.

3 Prompts are not security boundaries

Tool-using models may read untrusted web pages, documents, emails, or database fields. Indirect prompt injection can place adversarial instructions in this content and attempt to redirect a model's

actions [2]. Delimiters and instruction hierarchy can help a model reason about provenance, but text instructions alone cannot guarantee that untrusted content will not influence a stochastic component.

Defense in depth is more robust: sanitize or label untrusted content; minimize secret exposure; validate tool arguments with deterministic code; isolate browser/filesystem/network operations; require confirmation for side effects; log tool calls; and use egress controls. Treat retrieved text as data, not authority. If a workflow must act on data from an external source, validate the action against an independent policy rather than asking the model to police itself.

4 Monitoring, evaluation, and recovery

Control requires observability. Record model and prompt versions, tool schemas, input provenance, permissions, calls, outputs, approvals, costs, and exceptions. Logs enable incident response, reproducibility, and post-deployment evaluation, but logs themselves can contain sensitive content; access and retention need policy.

Test the actual workflow with a held-out suite. Include normal tasks, ambiguous tasks that should escalate, unauthorized action requests, malformed arguments, and prompt-injection payloads. Measure task success, false approvals, false blocks, intervention rate, time/cost, and severity of any violation. The NIST AI RMF frames risk management as a continuing process of governing, mapping, measuring, and managing risk rather than a one-time model test [3].

5 Human oversight is a design choice

Human-in-the-loop is valuable only if the person receives enough context and has meaningful authority. A reviewer confronted with hundreds of opaque approvals may rubber-stamp. Make proposed actions legible; show the target, diff, evidence, consequences, and alternatives; set deadlines appropriate to impact; and provide an immediate stop mechanism. For repetitive low-risk actions, sample audits and rate limits may be better than forcing manual clicks on every event.

Oversight also has failure modes: delayed response, automation bias, fatigue, and ambiguous responsibility. Design the workflow so that refusal or escalation is an accepted outcome, not a hidden penalty. The goal is calibrated delegation: automate what can be checked cheaply and reversibly, and retain stronger controls where an error is difficult to undo.

6 Worked control plan

Consider a research agent allowed to retrieve papers, write a draft bibliography, and propose code changes. It has read-only access to source archives and a sandboxed working directory. It may run tests locally, but cannot email, publish, alter credentials, or merge code. A human reviews a diff before a merge bot with narrowly scoped credentials acts. The test suite includes malicious instructions embedded in a retrieved PDF and checks that these do not create new authority.

7 Takeaway

Controllability is not a personality trait of an LLM. It is a system design and evidence problem: minimize authority, validate side effects deterministically, isolate untrusted inputs, monitor behavior, test boundaries, and make high-impact actions reversible or approval-gated.

References

[1] Anthropic. Model context protocol specification. *Official documentation*, 2024.

[2] Kai Greshake et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[3] National Institute of Standards and Technology. Ai risk management framework 1.0. *NIST AI 100-1*, 2023.